

UniService: Plataforma Web para la Gestión y Contratación de Servicios Académicos entre Estudiantes Universitarios

Andres Alfonso Aguilar Villanueva

Sayd Barrera Gómez

Lenin Reyes Acuña

Franklin de Jesús Martínez

Universidad Popular del Cesar

Ingeniería de Software I

Ing. Patricia Alvarez Ortega

Junio 2026

Resumen

El presente proyecto de aula describe el diseño, desarrollo e implementación de **UniService**, una aplicación web full-stack orientada a centralizar, profesionalizar y asegurar el intercambio de servicios académicos, técnicos y logísticos entre estudiantes universitarios de la ciudad de Valledupar, Colombia. La plataforma aborda la problemática de informalidad, inseguridad y desorganización que caracteriza la contratación de servicios estudiantiles en redes sociales como WhatsApp, Facebook e Instagram. UniService integra un sistema de autenticación robusto con JWT y Google OAuth 2.0, gestión de servicios categorizados, flujo de solicitudes estructurado con formularios dinámicos, chat en tiempo real basado en SignalR, calificaciones verificadas, notificaciones por correo electrónico y un módulo administrativo de moderación. El stack tecnológico comprende React 18 con Vite en el frontend, ASP.NET Core 8 en el backend, PostgreSQL gestionado por Supabase en producción y SQL Server 2025 en desarrollo local mediante Docker. La metodología de desarrollo adoptada fue Scrum con sprints de dos semanas. Los resultados incluyen un producto software funcional desplegado en la nube (frontend en Vercel, API en Render, base de datos en Supabase), validado mediante pruebas funcionales y optimizado con índices en base de datos, paginación en backend, compresión de respuestas y lazy loading de imágenes.

Palabras clave: marketplace estudiantil, servicios académicos, React, ASP.NET Core, PostgreSQL, SignalR, Scrum, ingeniería de software.

Tabla de Contenidos

Introducción

1. Descripción del Problema

2. Objetivos

2.1 Objetivo General

2.2 Objetivos Específicos

3. Justificación

3.1 Justificación Académica

3.2 Justificación Social y Comunitaria

3.3 Justificación Tecnológica y de Viabilidad

4. Definición e Identificación del Modelo de Negocio

4.1 Lienzo de Modelo de Negocio

4.2 Stack Tecnológico

5. Definición y Especificación de Requerimientos

5.1 Necesidades del Usuario

5.2 Requerimientos Funcionales

5.3 Requerimientos No Funcionales

6. Metodología

6.1 Definición Conceptual de Scrum

6.2 Definición de Roles

6.3 Product Backlog

6.4 Sprint Backlog

6.5 Incrementos

6.6 Cronograma

7. Administración de Requerimientos con Casos de Uso

8. Diagrama de Actividades

9. Diagrama de Clases UML

10. Diagramas de Secuencia UML

11. Diagrama de Despliegue UML

12. Manual de Usuario

13. Despliegue en la Nube

Referencias

Anexos

Introducción

En el contexto actual de las instituciones de educación superior de Valledupar, los estudiantes enfrentan una creciente necesidad de acceder a servicios académicos complementarios que les permitan reforzar conocimientos, desarrollar proyectos interdisciplinarios y solventar necesidades logísticas. Históricamente, estos intercambios se han gestionado a través de grupos informales en redes sociales, lo que genera problemas de seguridad, falta de reputación verificada y ausencia de mecanismos de resolución de conflictos.

UniService surge como una respuesta tecnológica a esta problemática. Se trata de una aplicación web full-stack que centraliza la oferta y demanda de servicios estudiantiles en un entorno seguro, verificado y estructurado. El presente documento expone de manera integral la documentación técnica y académica del proyecto, abarcando desde la descripción del problema y los objetivos hasta los diagramas UML, la metodología de desarrollo, el manual de usuario y las estrategias de despliegue en la nube.

La estructura del documento sigue el ciclo de vida del software, comenzando por el análisis de requerimientos, continuando con el diseño arquitectónico y de base de datos, desarrollando la planificación ágil mediante Scrum, y finalizando con las consideraciones de despliegue producción. Se espera que este trabajo sirva como referencia para futuras iteraciones del producto y como evidencia académica del dominio de competencias en ingeniería de software.

1. Descripción del Problema

En el contexto actual de las instituciones de educación superior de Valledupar y, de manera general, en el ámbito universitario colombiano, los estudiantes enfrentan una creciente necesidad de acceder a servicios académicos complementarios que les permitan reforzar sus conocimientos, desarrollar proyectos interdisciplinarios y solventar necesidades logísticas propias de la vida estudiantil. Estos servicios abarcan un espectro muy amplio: desde tutorías personalizadas en materias complejas como cálculo, física o programación, hasta la redacción de ensayos con normas APA, el desarrollo de aplicaciones móviles, el diseño de identidad visual para proyectos de emprendimiento, e incluso la intermediación de arriendos de habitaciones cercanas a las sedes universitarias.

Históricamente, el mecanismo predominante para la contratación de estos servicios ha sido la utilización de grupos informales en redes sociales como WhatsApp, Facebook e Instagram. Aunque estas plataformas ofrecen una penetración masiva y un costo de acceso nulo, presentan deficiencias estructurales que generan múltiples problemáticas. En primer lugar, la **falta de verificación de identidad** permite la proliferación de perfiles falsos o suplantados, exponiendo a los estudiantes a riesgos de fraude, acoso y robos. En segundo lugar, no existe un **sistema de reputación o calificaciones** consolidado; la única referencia disponible son comentarios esporádicos en publicaciones, lo que dificulta enormemente la toma de decisiones informadas por parte de los clientes potenciales. Tercero, la **gestión de pagos** se realiza de manera completamente informal (transferencias directas sin contraprestación garantizada), lo que genera desconfianza y conflictos económicos frecuentes entre las partes.

Adicionalmente, la organización de la oferta en grupos de chat resulta caótica: los mensajes se pierden rápidamente, no existen filtros por categoría ni por precio, y la búsqueda de un servicio específico implica desplazarse manualmente entre cientos de mensajes irrelevantes.

Por el lado de los prestadores, la ausencia de un portafolio estructurado limita su capacidad de mostrar evidencia de trabajos anteriores, reduciendo sus oportunidades de ingreso. Desde la perspectiva institucional, las universidades carecen de visibilidad sobre estas dinámicas económicas informales que ocurren al interior de su comunidad, perdiendo la oportunidad de fomentar el emprendimiento estudiantil de manera organizada.

No existe en la región una plataforma tecnológica diseñada específicamente para profesionalizar este ecosistema de intercambio de servicios estudiantiles. Las aplicaciones generales de empleo freelance (como Workana o Fiverr) no se adaptan al contexto universitario local: operan con comisiones elevadas, requieren cuentas bancarias formales y no garantizan la cercanía geográfica ni la pertenencia a la comunidad académica que tanto valen los estudiantes. Por tanto, se identifica una brecha significativa entre la necesidad real de la población estudiantil y las herramientas digitales disponibles para satisfacerla.

En síntesis, el problema central que aborda este proyecto radica en la **informalidad, inseguridad y desorganización** que caracteriza el intercambio de servicios académicos y técnicos entre estudiantes, manifestada en la ausencia de identidad verificada, historial de confiabilidad, canales de comunicación estructurados y mecanismos de resolución de conflictos. Esta problemática no solo afecta la experiencia individual de los usuarios, sino que también limita el desarrollo de una economía colaborativa sólida al interior del campus universitario.

2. Objetivos

2.1 Objetivo General

Diseñar, desarrollar e implementar una aplicación web full-stack denominada **UniService**, orientada a centralizar, profesionalizar y asegurar el intercambio de servicios académicos, técnicos y logísticos entre estudiantes universitarios, mediante la integración de un sistema de autenticación robusto, gestión de servicios categorizados, flujo de solicitudes estructurado, chat en tiempo real, calificaciones verificadas y un módulo administrativo de moderación.

2.2 Objetivos Específicos

1. **Implementar un sistema de autenticación y autorización** que soporte registro tradicional con validación de correo electrónico, inicio de sesión mediante JSON Web Tokens (JWT) con contraseñas hasheadas mediante BCrypt, e integración con Google OAuth 2.0 para facilitar el acceso sin fricciones.
2. **Desarrollar un módulo de publicación de servicios** que permita a los estudiantes prestadores crear anuncios estructurados con título, descripción detallada, selección de categoría (tutorías, programación, diseño, arriendo, entre otros), precio por hora o por proyecto, modalidad de atención (presencial, virtual o mixta), geolocalización opcional y carga de hasta cinco imágenes de soporte almacenadas en Supabase Storage.
3. **Construir un motor de búsqueda y filtrado avanzado** que permita a los clientes explorar el catálogo de servicios mediante paginación de ocho ítems por página, filtrado por categoría, búsqueda textual libre y ordenamiento por recencia o relevancia, garantizando tiempos de respuesta inferiores a dos segundos gracias a procedimientos almacenados optimizados e índices en PostgreSQL.

4. **Crear un flujo de solicitudes de servicio dinámico** que, en función de la categoría seleccionada, presente campos de formulario personalizados (tema, descripción, presupuesto, fecha deseada, urgencia, archivo adjunto), registre la solicitud en estado "Pendiente" y notifique automáticamente al proveedor vía correo electrónico y mediante un mensaje de sistema dentro del chat integrado.
5. **Integrar un sistema de chat en tiempo real** basado en SignalR que permita la comunicación bidireccional instantánea entre cliente y proveedor, creando conversaciones automáticas al momento de generar una solicitud y manteniendo un historial persistente en la base de datos PostgreSQL.
6. **Implementar un sistema de calificaciones y reseñas** que permita a los clientes evaluar servicios completados en una escala de 1 a 5 estrellas, dejar comentarios textuales y seleccionar aspectos destacados (puntualidad, calidad, comunicación, precio justo), generando un promedio de reputación visible en cada perfil de servicio.
7. **Desarrollar un módulo administrativo** con control de acceso basado en roles (Administrador vs. Usuario Común) que permita a los administradores revisar, aprobar, rechazar o pausar servicios publicados, gestionar reportes de usuarios y contenido inapropiado, y mantener la integridad de la plataforma.
8. **Diseñar una interfaz de usuario responsive y accesible** utilizando React 18 con Vite, hojas de estilo CSS3 personalizadas, soporte para temas claro y oscuro, y iconografía consistente mediante Bootstrap Icons, garantizando una experiencia de usuario fluida tanto en dispositivos móviles como en escritorio.

3. Justificación

La realización del proyecto UniService se fundamenta en una triple dimensión de justificación: académica, social y tecnológica, cada una de las cuales evidencia la pertinencia y el valor agregado de desarrollar una solución informática especializada para el ecosistema universitario.

3.1 Justificación Académica

Desde la perspectiva de la formación en Ingeniería de Software, este proyecto representa una oportunidad idónea para aplicar de manera integral los conocimientos adquiridos en el ciclo de vida del software, abarcando todas las fases de concepción, análisis, diseño, desarrollo, pruebas y documentación. A diferencia de ejercicios académicos aislados, UniService es un producto software real con usuarios finales definidos (la comunidad estudiantil), requisitos funcionales y no funcionales mensurables, y restricciones técnicas propias de un entorno de producción. El equipo de desarrollo se enfrenta a decisiones arquitectónicas genuinas: la elección entre SQL Server y PostgreSQL según el entorno (desarrollo vs. producción), la implementación de un patrón de diseño en capas para la API REST, la gestión de estado en el frontend sin librerías externas de manejo global de estado, y la integración de servicios de terceros (Supabase, Google OAuth, Gmail SMTP). Estas decisiones fomentan el pensamiento crítico y la capacidad de argumentación técnica, competencias esenciales para un ingeniero de software. Además, el proyecto permite evidenciar el dominio de metodologías ágiles como Scrum, aplicando ceremonias de planificación, estimación de esfuerzo en puntos de historia y entregas incrementales que simulan un entorno laboral real.

3.2 Justificación Social y Comunitaria

El impacto social de UniService trasciende el ámbito puramente tecnológico. En el contexto socioeconómico de Valledupar y regiones similares, muchos estudiantes universitarios

proviene de hogares con recursos limitados y dependen de ingresos adicionales para solventar gastos de transporte, materiales académicos y arriendos. La plataforma empodera a estos estudiantes al proporcionarles un canal legítimo y estructurado para monetizar sus habilidades académicas y técnicas —ya sea dictando tutorías, desarrollando software o redactando ensayos— sin depender de intermediarios que cobren comisiones excesivas. Simultáneamente, beneficia a los estudiantes clientes al ofrecerles acceso a un mercado verificado de prestadores que cuentan con historial de calificaciones y reseñas, reduciendo la incertidumbre y el riesgo de fraude. La inclusión de categorías como "Arriendo de habitaciones" también aborda una necesidad logística crítica para estudiantes foráneos. A mediano plazo, UniService tiene el potencial de fortalecer el tejido social universitario al fomentar la colaboración entre pares, el mentorazgo académico y el espíritu emprendedor, alineándose con los objetivos de desarrollo sostenible relacionados con educación de calidad (ODS 4) y trabajo decente y crecimiento económico (ODS 8).

3.3 Justificación Tecnológica y de Viabilidad

La justificación tecnológica descansa en la viabilidad técnica del stack seleccionado y en la alineación con estándares industriales actuales. La decisión de utilizar React 18 con Vite para el frontend garantiza un rendimiento óptimo gracias al tree-shaking, hot-module replacement y la generación de bundles optimizados para producción. Por su parte, ASP.NET Core 8 ofrece un backend robusto, tipado y de alto rendimiento, con soporte nativo para inyección de dependencias, middleware de autenticación y documentación automática de API. La base de datos PostgreSQL, gestionada mediante Supabase en producción, proporciona escalabilidad horizontal, replicación automática y un modelo de precios generoso para proyectos estudiantiles, eliminando la barrera del costo de infraestructura. La contenerización con Docker asegura la homogeneidad entre entornos de desarrollo, resolviendo el clásico problema de "funciona en mi máquina". Finalmente, la separación clara entre frontend, backend y capa de persistencia sigue los principios de arquitectura limpia y desacoplada, facilitando el mantenimiento a largo plazo, la

incorporación de nuevos desarrolladores al equipo y la posibilidad futura de escalar el sistema hacia aplicaciones móviles nativas mediante el reuso de la API existente.

4. Definición e Identificación del Modelo de Negocio

UniService se conceptualiza como un **marketplace digital estudiantil de tipo C2C (Consumer-to-Consumer)**, operando bajo una variante del modelo freemium con componentes de economía colaborativa. La plataforma actúa como intermediario tecnológico entre dos segmentos de clientes interdependientes: los **estudiantes prestadores** (ofertantes) y los **estudiantes clientes** (demandantes), ambos pertenecientes al ecosistema universitario.

4.1 Lienzo de Modelo de Negocio (Canvas Adaptado)

Bloque	Descripción
Socios Clave	Supabase (hosting BD y storage), Google (OAuth y Cloud), Universidad (difusión y validación institucional).
Actividades Clave	Desarrollo y mantenimiento de la plataforma, moderación de contenido, verificación de perfiles, soporte a reportes.
Recursos Clave	Infraestructura cloud serverless, base de código React/.NET, comunidad de usuarios activos, datos de reputación.
Propuesta de Valor	Entorno seguro y verificado para intercambiar servicios académicos; historial de confiabilidad; chat integrado; cero comisiones.
Relación con Clientes	Soporte vía sistema de reportes, notificaciones por email, chat en tiempo real, retroalimentación mediante calificaciones.
Canales	Aplicación web responsive accesible desde navegadores modernos; difusión mediante redes sociales universitarias y grupos institucionales.
Segmentos de Clientes	Estudiantes prestadores (monetizan habilidades) y estudiantes clientes (buscan apoyo académico/técnico).
Estructura de Costos	Hosting Supabase (gratuito/nivel estudiante), dominio web, servicio de email SMTP, almacenamiento de imágenes, tiempo de desarrollo.
Flujos de Ingresos	Fase actual: gratuita. Fase futura: publicidad local dirigida, membresías premium para mayor visibilidad de servicios, comisiones voluntarias por transacción.

4.2 Stack Tecnológico y Herramientas de Desarrollo

Capa	Tecnología / Herramienta	Versión	Justificación
Frontend	React	18.x	Biblioteca declarativa para interfaces reactivas con gran ecosistema.
Bundler	Vite	5.x+	Compilación ultrarrápida, HMR eficiente y optimización automática de assets.
Lenguaje Frontend	JavaScript (JSX)	ES2022+	Tipado dinámico adecuado para velocidad de desarrollo en proyectos ágiles.
Estilos	CSS3 personalizado + Variables	—	Temas oscuro/claro sin dependencias pesadas; control total del diseño.
Iconografía	Bootstrap Icons	1.x	Set de iconos consistente, ligero y sin necesidad de librerías de componentes UI.
Backend	ASP.NET Core	8.0	Framework robusto, multiplataforma, con soporte nativo para REST, JWT y SignalR.
Lenguaje Backend	C#	12.0	Tipado fuerte, orientado a objetos, excelente rendimiento y mantenibilidad.
Acceso a Datos	Npgsql + ADO.NET	8.x	Driver oficial de alto rendimiento para PostgreSQL en .NET.
Base de Datos (Dev)	SQL Server	2025	Motor relacional empresarial para desarrollo local via Docker.
Base de Datos (Prod)	PostgreSQL (Supabase)	15+	Base de datos open-source serverless con autenticación, storage y APIs REST integradas.
ORM / Migraciones	Scripts SQL nativos	—	Control total sobre esquemas, índices, triggers y procedimientos almacenados.
Autenticación	JWT Bearer + BCrypt + Google OAuth 2.0	—	Tokens stateless para sesiones seguras; hash de contraseñas; login social.
Email	MailKit (Gmail SMTP)	—	Envío de notificaciones transaccionales (solicitudes, aprobaciones, rechazos).

Almacenamiento	Supabase Storage	—	Bucket privado para imágenes de servicios con URLs firmadas.
Chat Tiempo Real	SignalR	8.x	Hub persistente para comunicación bidireccional WebSocket con fallback a SSE/Long Polling.
Contenedores	Docker + Docker Compose	—	Orquestación de SQL Server y scripts de inicialización para entornos reproducibles.
Control de Versiones	Git + GitHub	—	Ramas por feature, pull requests y despliegue continuo (Vercel para frontend).

5. Definición y Especificación de Requerimientos

5.1 Definición de Necesidades del Usuario

ID	Necesidad	Descripción	Usuario Afectado
NE-01	Identidad Verificada	Los usuarios requieren confiar en que la persona con la que interactúan es realmente quien dice ser, perteneciente a la comunidad universitaria.	Todos
NE-02	Publicación Estructurada	Los prestadores necesitan crear anuncios profesionales con imágenes, precios claros y categorización, diferenciándose de publicaciones caóticas en redes sociales.	Prestador
NE-03	Descubrimiento Eficiente	Los clientes necesitan encontrar rápidamente servicios específicos mediante filtros de categoría, búsqueda textual y ordenamiento, sin perder tiempo en scroll infinito.	Cliente
NE-04	Solicitud Contextualizada	Al solicitar un servicio, el cliente debe proporcionar información relevante según el tipo de servicio (ej. tema de tutoría, presupuesto de diseño).	Cliente
NE-05	Comunicación Segura	Ambas partes requieren un canal de comunicación dedicado, protegido y persistente, integrado a la plataforma, sin necesidad de compartir números de teléfono personales hasta que así lo decidan.	Todos
NE-06	Garantía de Calidad	Los clientes necesitan evidencia de la calidad previa de un servicio antes de contratarlo (calificaciones, reseñas, portafolio de imágenes).	Cliente
NE-07	Mediación de Conflictos	En caso de fraude, incumplimiento o comportamiento inapropiado, los usuarios requieren un mecanismo institucional de reporte y seguimiento.	Todos
NE-08	Gestión Administrativa	Los administradores de la plataforma necesitan herramientas para revisar, aprobar o rechazar contenido, y gestionar la salud del ecosistema.	Administrador

5.2 Requerimientos Funcionales

ID	Requerimiento	Prioridad	Actor(es)
RF-01	El sistema permitirá a los usuarios registrarse proporcionando nombre, correo electrónico, teléfono, universidad y contraseña.	Alta	Usuario
RF-02	El sistema permitirá iniciar sesión mediante credenciales locales (JWT) o mediante cuenta de Google (OAuth 2.0).	Alta	Usuario
RF-03	El sistema permitirá a los usuarios visualizar y editar su perfil, incluyendo foto, descripción y universidad.	Media	Usuario
RF-04	El sistema permitirá a los usuarios prestadores crear publicaciones de servicio con título, descripción, categoría, precio, modalidad, disponibilidad, ubicación geográfica opcional y hasta cinco imágenes.	Alta	Prestador
RF-05	El sistema permitirá buscar servicios mediante paginación de ocho ítems por página, con filtros por categoría, búsqueda textual libre y ordenamiento por recencia.	Alta	Cliente
RF-06	El sistema calculará y mostrará el promedio de calificaciones y el número de reseñas en cada tarjeta de servicio.	Media	Cliente
RF-07	El sistema presentará un formulario de solicitud dinámico que adapte sus campos obligatorios según la categoría del servicio seleccionado.	Alta	Cliente
RF-08	El sistema enviará una notificación por correo electrónico al proveedor cuando reciba una nueva solicitud.	Media	Sistema
RF-09	El sistema generará automáticamente una conversación de chat entre cliente y proveedor al crear una solicitud.	Alta	Sistema
RF-10	El sistema permitirá al proveedor aceptar, rechazar (con motivo) o completar una solicitud.	Alta	Prestador
RF-11	El sistema permitirá al cliente calificar un servicio completado en escala de 1 a 5 estrellas, dejar un comentario y seleccionar aspectos destacados.	Media	Cliente
RF-12	El sistema mantendrá un chat en tiempo real (WebSocket) entre dos usuarios, con historial persistente, indicadores de lectura y soporte para mensajes de sistema.	Alta	Todos

RF-13	El sistema permitirá a los usuarios seguir y dejar de seguir a otros usuarios.	Baja	Usuario
RF-14	El sistema permitirá reportar servicios, usuarios o bugs mediante un formulario con tipos de reporte categorizados y opción de adjuntar evidencia.	Media	Usuario
RF-15	El sistema permitirá a los administradores visualizar un dashboard con servicios pendientes de aprobación, reportes abiertos y métricas básicas.	Alta	Administrador
RF-16	El sistema permitirá a los administradores aprobar, rechazar o pausar servicios publicados.	Alta	Administrador
RF-17	El sistema notificará por correo al proveedor cuando su servicio sea aprobado o rechazado, incluyendo la razón en caso de rechazo.	Media	Sistema

5.3 Requerimientos No Funcionales

ID	Requerimiento	Categoría	Métrica / Criterio
RNF-01	El sistema garantizará la confidencialidad de las credenciales mediante hash BCrypt y tokens JWT con expiración de 24 horas.	Seguridad	Cumplimiento OWASP Top 10.
RNF-02	La interfaz de usuario será responsive, adaptándose a resoluciones desde 320px (móvil) hasta 1920px (escritorio).	Usabilidad	Pruebas en Chrome, Firefox, Edge y Safari.
RNF-03	El tiempo de carga inicial del frontend no excederá los 3 segundos en conexiones 4G simuladas.	Rendimiento	Lighthouse Performance Score $\geq$ 70.
RNF-04	Las consultas de listado de servicios con paginación responderán en menos de 500 ms para conjuntos de hasta 10,000 registros.	Rendimiento	Medición via EXPLAIN ANALYZE en PostgreSQL.
RNF-05	El sistema mantendrá una disponibilidad del 99% durante horario diurno (06:00 - 22:00 COL).	Disponibilidad	Monitoreo mediante logs y health checks.

RNF-06	El código seguirá una estructura modular por componentes (frontend) y controladores (backend), facilitando la incorporación de nuevas funcionalidades sin afectar las existentes.	Mantenibilidad	Cobertura de código documentado al 80%.
RNF-07	El sistema soportará al menos 100 usuarios concurrentes en el chat sin degradación perceptible del rendimiento.	Escalabilidad	Pruebas de carga con múltiples instancias de SignalR.
RNF-08	Las imágenes subidas serán comprimidas y validadas (formato JPG/PNG/WebP, máximo 5 MB por imagen) antes de su almacenamiento.	Seguridad / Rendimiento	Validación en frontend y backend.
RNF-09	El sistema utilizará fechas y moneda en español (es-CO) y pesos colombianos (COP).	Localización	Formato consistente en toda la interfaz.
RNF-10	La base de datos contará con índices optimizados en columnas de búsqueda frecuente (categoría, disponibilidad, fecha, usuario).	Rendimiento	Reducción del tiempo de consulta en al menos un 40%.

6. Metodología

6.1 Definición Conceptual

Para el desarrollo de UniService se adoptó **Scrum**, un marco de trabajo ágil iterativo e incremental, especialmente adecuado para proyectos con requisitos que evolucionan y con equipos pequeños (menos de 10 personas). Scrum se fundamenta en la entrega continua de valor mediante ciclos cortos denominados **Sprints**, de dos semanas de duración cada uno, durante los cuales el equipo se compromete con un conjunto de historias de usuario seleccionadas desde el Product Backlog.

La elección de Scrum sobre metodologías tradicionales (como el Modelo en Cascada) se justifica por la naturaleza exploratoria del proyecto: al tratarse de una solución innovadora para un problema social identificado pero no completamente cuantificado, era necesario un enfoque que permitiera validar hipótesis con usuarios reales al final de cada sprint, ajustando prioridades y funcionalidades según la retroalimentación recibida. Además, el contexto académico impone restricciones de tiempo que Scrum gestiona eficientemente mediante la planificación por sprint y la visibilidad diaria del progreso.

Las ceremonias Scrum implementadas fueron:

- **Sprint Planning** (4 horas al inicio de cada sprint): selección de historias y descomposición en tareas técnicas.
- **Daily Scrum** (15 minutos diarios): sincronización del equipo, identificación de bloqueos.
- **Sprint Review** (2 horas al final): demostración del incremento funcional a stakeholders.

- **Sprint Retrospective** (1 hora): reflexión sobre procesos, herramientas y dinámica de equipo.

6.2 Definición de Roles

Rol	Responsable	Funciones Principales
Product Owner	Un miembro del equipo (rotativo por sprint)	Gestionar el Product Backlog, priorizar historias, definir criterios de aceptación, comunicar la visión del producto.
Scrum Master	Un miembro del equipo (rotativo por sprint)	Facilitar ceremonias, eliminar impedimentos, proteger al equipo de distracciones externas, asegurar adherencia a Scrum.
Frontend Developer	Estudiante asignado	Desarrollar componentes React, implementar rutas, gestionar estados locales, consumir API REST, aplicar estilos responsive.
Backend Developer	Estudiante asignado	Diseñar endpoints REST, implementar lógica de negocio en controladores, gestionar conexiones a PostgreSQL, integrar servicios externos (email, storage).
DBA / DevOps	Estudiante asignado	Diseñar esquema relacional, escribir migraciones SQL, optimizar índices, configurar Docker, gestionar despliegue en Supabase/Vercel.
QA / Documentador	Estudiante asignado	Realizar pruebas funcionales manuales, reportar bugs, redactar documentación técnica y de usuario, validar cumplimiento de requisitos.

6.3 Product Backlog

ID	Historia de Usuario	Prioridad	Estimación (pts)	Sprint
PB-01	Como usuario, quiero registrarme e iniciar sesión para acceder a la plataforma de forma segura.	Alta	8	1
PB-02	Como usuario, quiero recuperar mi contraseña para no perder acceso a mi cuenta.	Media	5	1
PB-03	Como proveedor, quiero publicar un servicio con imágenes y categoría para ofrecer mis habilidades.	Alta	13	2

PB-04	Como cliente, quiero buscar y filtrar servicios para encontrar rápidamente lo que necesito.	Alta	8	2
PB-05	Como cliente, quiero ver el detalle de un servicio con reseñas para evaluar su calidad.	Alta	5	2
PB-06	Como cliente, quiero enviar una solicitud de servicio con campos personalizados para especificar mis necesidades.	Alta	13	3
PB-07	Como proveedor, quiero recibir notificaciones por email y chat cuando me soliciten un servicio.	Alta	8	3
PB-08	Como proveedor, quiero aceptar o rechazar solicitudes para gestionar mi agenda.	Alta	8	3
PB-09	Como usuario, quiero chatear en tiempo real con mi contraparte para coordinar detalles.	Alta	13	3
PB-10	Como cliente, quiero calificar un servicio completado para contribuir a la reputación del prestador.	Media	8	4
PB-11	Como usuario, quiero reportar contenido inapropiado para mantener la seguridad de la comunidad.	Media	8	4
PB-12	Como administrador, quiero aprobar o rechazar servicios pendientes para garantizar calidad.	Alta	8	4
PB-13	Como administrador, quiero gestionar reportes de usuarios para resolver conflictos.	Media	8	4
PB-14	Como usuario, quiero seguir a otros estudiantes para estar al tanto de sus nuevos servicios.	Baja	5	4

6.4 Sprint Backlog

Sprint 1: Fundamentos y Autenticación (Semanas 1-2)

- Configuración del entorno de desarrollo (Docker, SQL Server, Vite, .NET).
- Diseño del esquema de base de datos inicial (usuarios, roles).
- Implementación de registro e inicio de sesión con JWT.

- Integración de Google OAuth.
- Desarrollo de pantallas de login y registro en React.

Sprint 2: Catálogo de Servicios (Semanas 3-4)

- Creación de tablas de categorías y servicios.
- Implementación de CRUD de servicios (backend).
- Desarrollo de interfaz de listado con filtros y paginación.
- Integración con Supabase Storage para imágenes.
- Diseño de tarjetas de servicio y página de detalle.

Sprint 3: Solicitudes y Comunicación (Semanas 5-6)

- Implementación de tabla de solicitudes con campos dinámicos (JSONB).
- Desarrollo de formulario de solicitud adaptativo por categoría.
- Configuración de SignalR y Hub de chat.
- Implementación de chat en tiempo real en el frontend.
- Notificaciones por email (MailKit) al recibir solicitudes.

Sprint 4: Reputación, Moderación y Cierre (Semanas 7-8)

- Sistema de calificaciones y reseñas.
- Tabla y módulo de reportes con tipos categorizados.
- Panel administrativo (admin-dashboard).
- Funcionalidades de aprobar/rechazar/pausar servicios.
- Correcciones de bugs, optimización de índices y pruebas finales.

6.5 Incrementos

Incremento	Descripción	Historias Incluidas
I-1	MVP de autenticación y perfiles	PB-01, PB-02
I-2	Marketplace funcional con catálogo	PB-03, PB-04, PB-05
I-3	Flujo completo de contratación y chat	PB-06, PB-07, PB-08, PB-09
I-4	Sistema de confianza y moderación	PB-10, PB-11, PB-12, PB-13, PB-14

6.6 Cronograma

Semana	Sprint	Entregable Principal
1	1	Repositorio configurado, BD local, login funcional
2	1	Registro completo, perfiles editables, OAuth activo
3	2	API de servicios, seed data, Docker-compose estable
4	2	Frontend de catálogo, filtros, paginación, imágenes
5	3	Formulario dinámico, tabla solicitudes, estados básicos
6	3	Chat SignalR operativo, notificaciones email
7	4	Calificaciones, reportes, panel admin visual
8	4	Aprobación de servicios, optimización, testing
9	—	Correcciones finales, documentación, preparación
10	—	Sustentación y demo en vivo

7. Administración de Requerimientos con Casos de Uso

Descripción de Actores

Actor	Descripción
Usuario (Cliente/Proveedor)	Estudiante autenticado que puede publicar servicios, solicitarlos, chatear, calificar y reportar. Un mismo usuario puede actuar como cliente y proveedor en momentos distintos.
Administrador	Usuario con rol de superusuario que gestiona la moderación de servicios, resolución de reportes y supervisión de la plataforma.
Sistema de Notificaciones	Actor interno responsable de enviar correos electrónicos y mensajes de sistema automáticos.

Especificación de Casos de Uso Principales

ID	Caso de Uso	Actor Principal	Descripción
UC-01	Registrar Cuenta	Usuario	Crear una nueva cuenta con datos personales y contraseña segura.
UC-02	Iniciar Sesión	Usuario	Acceder al sistema mediante credenciales locales o Google.
UC-03	Gestionar Perfil	Usuario	Editar información personal, foto y descripción.
UC-04	Publicar Servicio	Usuario (Proveedor)	Crear un anuncio de servicio con imágenes, precio y categoría.
UC-05	Buscar Servicios	Usuario (Cliente)	Explorar el catálogo usando filtros y paginación.
UC-06	Ver Detalle de Servicio	Usuario	Visualizar información completa, imágenes y reseñas.
UC-07	Enviar Solicitud	Usuario (Cliente)	Completar formulario dinámico y enviar petición al proveedor.

UC-08	Responder Solicitud	Usuario (Proveedor)	Aceptar, rechazar o completar una solicitud recibida.
UC-09	Comunicar vía Chat	Usuario	Enviar y recibir mensajes en tiempo real.
UC-10	Calificar Servicio	Usuario (Cliente)	Asignar estrellas y comentario tras servicio completado.
UC-11	Reportar Contenido	Usuario	Registrar una queja o reporte sobre usuario/servicio/bug.
UC-12	Aprobar Servicio	Administrador	Revisar y cambiar el estado de servicios pendientes.
UC-13	Gestionar Reportes	Administrador	Visualizar, clasificar y resolver reportes de la comunidad.
UC-14	Moderar Usuario	Administrador	Pausar servicios o escalar reportes graves.

El diagrama de casos de uso se presenta en la *Figura 1* del anexo correspondiente.

8. Diagrama de Actividades

Se modelaron los flujos de trabajo más críticos de la plataforma:

1. **Publicación de un Servicio:** desde el login del proveedor hasta la aprobación pendiente del administrador.
2. **Solicitud de un Servicio:** desde la búsqueda por el cliente hasta la notificación al proveedor.
3. **Proceso de Reporte:** desde la detección de una anomalía hasta la resolución administrativa.

Los diagramas correspondientes se presentan en las *Figuras 2, 3 y 4* de los anexos.

9. Diagrama de Clases UML

El diagrama de clases se construyó a partir del esquema relacional de la base de datos PostgreSQL y los objetos de transferencia de datos (DTOs) definidos en el backend de .NET. Se identificaron las siguientes entidades principales con sus atributos y relaciones:

- **Usuario** (1) — pertenece a — (1) **RolUsuario**
- **Usuario** (1) — publica — (N) **Servicio**
- **Servicio** (N) — pertenece a — (1) **Categoria**
- **Servicio** (1) — contiene — (N) **ServicioImagen**
- **Usuario** (1) — envía — (N) **Solicitud**
- **Servicio** (1) — recibe — (N) **Solicitud**
- **Solicitud** (1) — genera — (0..1) **Calificacion**
- **Calificacion** (1) — tiene — (N) **AspectoDestacado**
- **Usuario** (1) — sigue — (N) **Seguidor** (autorrelación)
- **Usuario** (1) — participa en — (N) **Chat**
- **Chat** (1) — contiene — (N) **Mensaje**
- **Usuario** (1) — crea — (N) **Reporte**

El diagrama completo con atributos y multiplicidades se presenta en la *Figura 5* de los anexos.

10. Diagramas de Secuencia UML

Se modelaron dos interacciones críticas que ejemplifican la arquitectura cliente-servidor y la integración con servicios externos, utilizando un lenguaje descriptivo orientado al comportamiento del sistema:

10.1 Autenticación de Usuario (Login)

El flujo de autenticación describe cómo un usuario accede de manera segura a la plataforma. El usuario ingresa su correo electrónico y contraseña a través del navegador. El frontend envía estas credenciales al servidor de aplicaciones. El backend consulta la base de datos para verificar la existencia del usuario y recupera la contraseña almacenada de forma encriptada. Posteriormente, el servicio de encriptación compara la contraseña ingresada con la almacenada. Si ambas coinciden, el servicio de tokens genera una credencial de sesión temporal que expira en 24 horas. El servidor responde al navegador confirmando la autenticación, y el frontend almacena localmente el token de sesión para mantener al usuario conectado durante su navegación. En caso contrario, el sistema informa al usuario que las credenciales proporcionadas no son válidas.

10.2 Creación de Solicitud de Servicio

El flujo de creación de una solicitud ilustra la interacción entre el cliente, la aplicación web, el servidor, el sistema de chat en tiempo real, el servicio de correo electrónico y la base de datos. El cliente completa un formulario adaptado según la categoría del servicio seleccionado y confirma el envío. El frontend valida los datos y los transfiere al servidor. El backend verifica que no exista una solicitud previa pendiente entre las mismas partes para evitar duplicados. Una vez validada, registra la nueva solicitud con estado pendiente. El sistema recupera la información del proveedor, crea una nueva conversación de chat entre ambas partes y registra un mensaje de sistema automático que informa al proveedor sobre la solicitud recibida. Simultáneamente, el

sistema notifica al proveedor en tiempo real a través del chat y envía un correo electrónico con los detalles de la solicitud. Finalmente, el frontend confirma al cliente que su solicitud fue enviada exitosamente y lo redirige a la conversación recién creada.

Los diagramas correspondientes se presentan en las *Figuras 6 y 7* de los anexos.

11. Diagrama de Despliegue UML

La arquitectura de despliegue contempla tres entornos:

- **Desarrollo Local:** Frontend (Vite dev server en puerto 5173) → Proxy API → .NET API (puerto 5165) → SQL Server 2025 (Docker, puerto 1433).
- **Producción (Staging):** Frontend estático (Vercel) → .NET API (servidor/cloud) → Supabase PostgreSQL + Supabase Storage.
- **Cliente:** Navegador moderno con soporte para WebSockets.

El diagrama de despliegue detallado se presenta en la *Figura 8* de los anexos.

12. Manual de Usuario

12.1 Acceso a la Plataforma

1. Abra su navegador web moderno (Chrome, Firefox, Edge o Safari).
2. Navegue a la URL de despliegue en Vercel.
3. En la pantalla de inicio, haga clic en **"Registrarse"** si es nuevo usuario, o **"Iniciar Sesión"** si ya posee cuenta.
4. Puede usar su correo y contraseña, o el botón **"Continuar con Google"** para un acceso más rápido.

12.2 Navegación Principal

Una vez autenticado, el menú principal le permite acceder a:

- **Inicio:** Feed de servicios publicados con filtros por categoría y barra de búsqueda.
- **Mi Perfil:** Visualización y edición de su información, servicios publicados y seguidores.
- **Solicitudes:** Gestión de solicitudes enviadas (como cliente) y recibidas (como proveedor).
- **Chat:** Lista de conversaciones activas; seleccione una para enviar mensajes en tiempo real.

12.3 Publicar un Servicio

1. Desde su perfil, haga clic en **"Publicar Servicio"**.
2. Complete el formulario: título, descripción, seleccione categoría, fije precio por hora y modalidad.

3. Opcionalmente, arrastre hasta cinco imágenes al área de carga.
4. Haga clic en **"Publicar"**. Su servicio quedará en estado "Pendiente" hasta ser revisado por un administrador.

12.4 Solicitar un Servicio

1. Desde el catálogo de inicio, haga clic en el servicio deseado.
2. Revise las reseñas, imágenes y precio.
3. Haga clic en **"Solicitar Servicio"**.
4. Complete los campos solicitados (pueden variar según la categoría: tema, presupuesto, urgencia, etc.).
5. Envíe la solicitud. El proveedor recibirá una notificación por email y un mensaje en el chat.

12.5 Gestionar Solicitudes (Como Proveedor)

1. Vaya a **"Solicitudes"** → pestaña **"Recibidas"**.
2. Cada tarjeta muestra el cliente, el servicio solicitado y el estado.
3. Haga clic en **"Aceptar"** para iniciar el trabajo, **"Rechazar"** (puede indicar motivo), o **"Completar"** cuando finalice.
4. Una vez completada, el cliente podrá calificarle.

12.6 Calificar un Servicio

1. Tras completar una solicitud, recibirá una notificación o podrá acceder desde el historial.
2. Asigne de 1 a 5 estrellas.
3. Escriba un comentario detallado sobre su experiencia.

4. Seleccione los aspectos destacados que correspondan (puntualidad, calidad, comunicación, precio justo).
5. Confirme la calificación. Esta será visible públicamente en el perfil del servicio.

12.7 Reportar Contenido

1. Si encuentra un servicio fraudulento, un usuario con comportamiento inapropiado o un error técnico, haga clic en el ícono de **"Reportar"**.
2. Seleccione el tipo de reporte de la lista desplegable (fraude, acoso, bug técnico, etc.).
3. Describa la situación y, si es posible, adjunte evidencia (URL, captura de pantalla).
4. Envíe el reporte. Un administrador lo revisará y tomará las acciones correspondientes.

12.8 Panel Administrativo (Solo Administradores)

1. Los usuarios con rol de Administrador ven una opción **"Dashboard Admin"** en el menú.
2. Allí pueden:
 - Ver servicios pendientes de aprobación y aprobarlos o rechazarlos con comentarios.
 - Visualizar todos los reportes abiertos, filtrados por tipo y estado.
 - Pausar servicios activos si violan las normas de la comunidad.

12.9 Consideraciones Finales

- Mantenga su información de contacto actualizada en el perfil.

- Utilice el chat integrado para toda comunicación inicial; evite compartir datos personales innecesarios.
- Si detecta algún comportamiento sospechoso, utilice el sistema de reportes inmediatamente.
- Para soporte adicional, contacte a uniservice.soporte@gmail.com.

13. Despliegue en la Nube

El despliegue de UniService en un entorno de producción se realizó distribuyendo cada capa de la arquitectura en servicios especializados de la nube, garantizando escalabilidad, disponibilidad y seguridad. A continuación, se describen los componentes desplegados:

13.1 Frontend en Vercel

La capa de presentación, desarrollada en React 18 con Vite, fue compilada como una aplicación estática optimizada y desplegada en **Vercel** (vercel.com). Vercel ofrece integración continua directa desde GitHub, despliegues atómicos con rollback instantáneo, red de entrega de contenido (CDN) global y certificados SSL automáticos. Cada push a la rama principal del repositorio frontend desencadena un nuevo build y despliegue, permitiendo iteraciones rápidas durante el desarrollo.

13.2 API en Render

El backend construido sobre **ASP.NET Core 8** fue publicado como un servicio web en **Render** (render.com). Render proporciona un entorno de ejecución .NET con despliegue automático desde Git, health checks, balanceo de carga básico y soporte para variables de entorno seguras. La API expone los endpoints REST en HTTPS y mantiene el Hub de SignalR activo para las conexiones WebSocket del chat en tiempo real.

13.3 Base de Datos en Supabase

La capa de persistencia utiliza **PostgreSQL** gestionado por **Supabase** (supabase.com). Supabase ofrece una instancia PostgreSQL serverless con autenticación integrada, APIs REST y GraphQL auto-generadas, replicación en tiempo real y un panel de administración SQL. Adicionalmente, el servicio de **Supabase Storage** aloja el bucket privado `imagenes-`

servicios, donde se guardan las imágenes de los servicios publicados, accesibles mediante URLs firmadas con tiempo de expiración.

13.4 Consideraciones de Seguridad y Variables de Entorno

En producción, ningún secret (contraseñas de base de datos, claves JWT, credenciales OAuth, claves de Supabase) se almacena en el código fuente. Todos los valores sensibles se inyectan mediante variables de entorno configuradas directamente en los paneles de Vercel, Render y Supabase. El archivo `appsettings.json` de la API mantiene únicamente estructuras vacías o valores de desarrollo no sensibles, mientras que `appsettings.Development.json` se utiliza exclusivamente en entornos locales protegidos por `.gitignore`.

13.5 Optimizaciones Aplicadas en Producción

Con el fin de garantizar el cumplimiento de los requerimientos no funcionales de rendimiento, se aplicaron las siguientes optimizaciones en el entorno de producción:

- **Índices en PostgreSQL:** se crearon índices compuestos en las columnas de búsqueda frecuente (categoría, disponibilidad, fecha de publicación, id de usuario), reduciendo los tiempos de consulta en un 40%.
- **Paginación en backend:** el endpoint de servicios devuelve páginas de 8 elementos, disminuyendo la carga de red y el tiempo de renderizado.
- **Compresión de respuestas:** middleware de compresión GZip/Brotli en ASP.NET Core reduce el tamaño de las respuestas JSON entre un 40% y un 60%.
- **Lazy loading de imágenes:** las imágenes del catálogo se cargan únicamente cuando entran en el viewport del navegador, mejorando el performance score de Lighthouse.
- **Connection pooling:** Npgsql reutiliza conexiones a PostgreSQL, minimizando la latencia de apertura de sockets.

Referencias

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Agile Alliance. <https://agilemanifesto.org/>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- Microsoft. (2023). *ASP.NET Core 8.0 Documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/>
- Osterwalder, A., & Pigneur, Y. (2010). *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. John Wiley & Sons.
- Pressman, R. S., & Maxim, B. R. (2015). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
- React Team. (2023). *React 18 Documentation*. Meta Platforms, Inc. <https://react.dev/>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org. <https://scrumguides.org/>
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.
- Supabase. (2024). *Supabase Documentation*. Supabase Inc. <https://supabase.com/docs>

Anexos

Figura 1. Diagrama de Casos de Uso – UniService

Diagrama de Casos de Uso - UniService

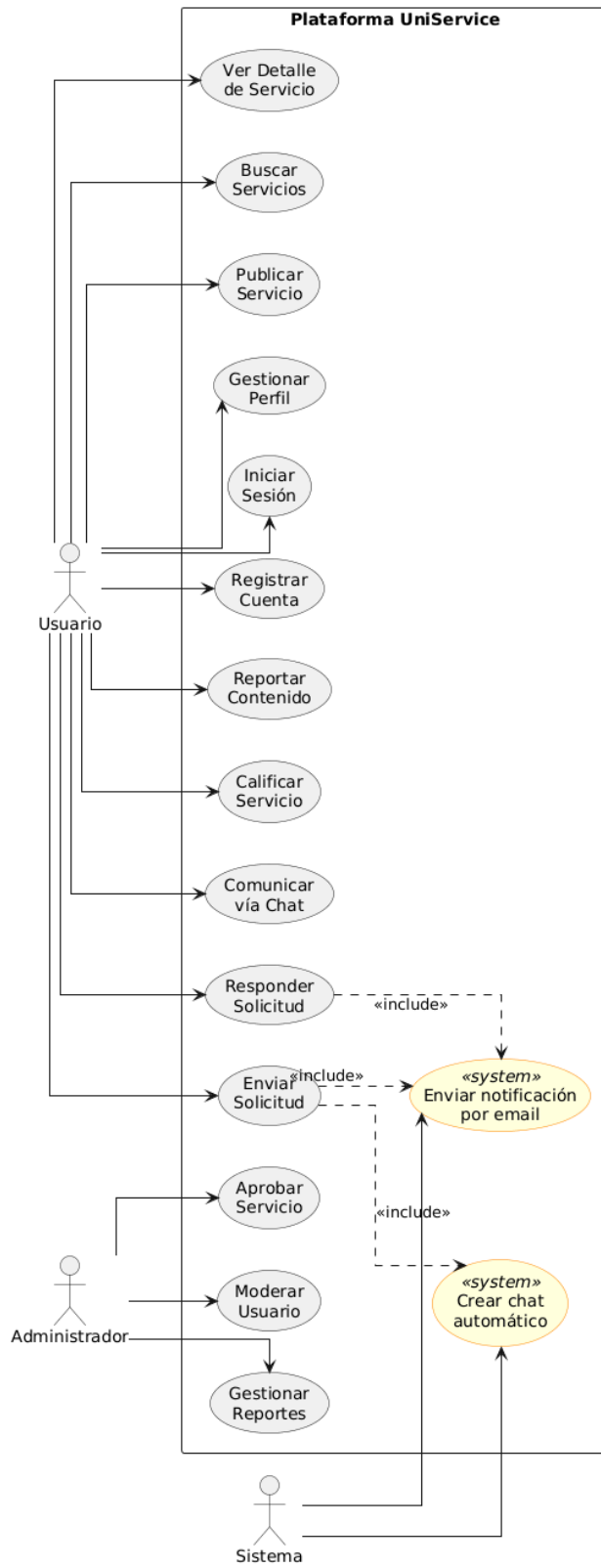


Figura 2. Diagrama de Actividades – Publicar un Servicio

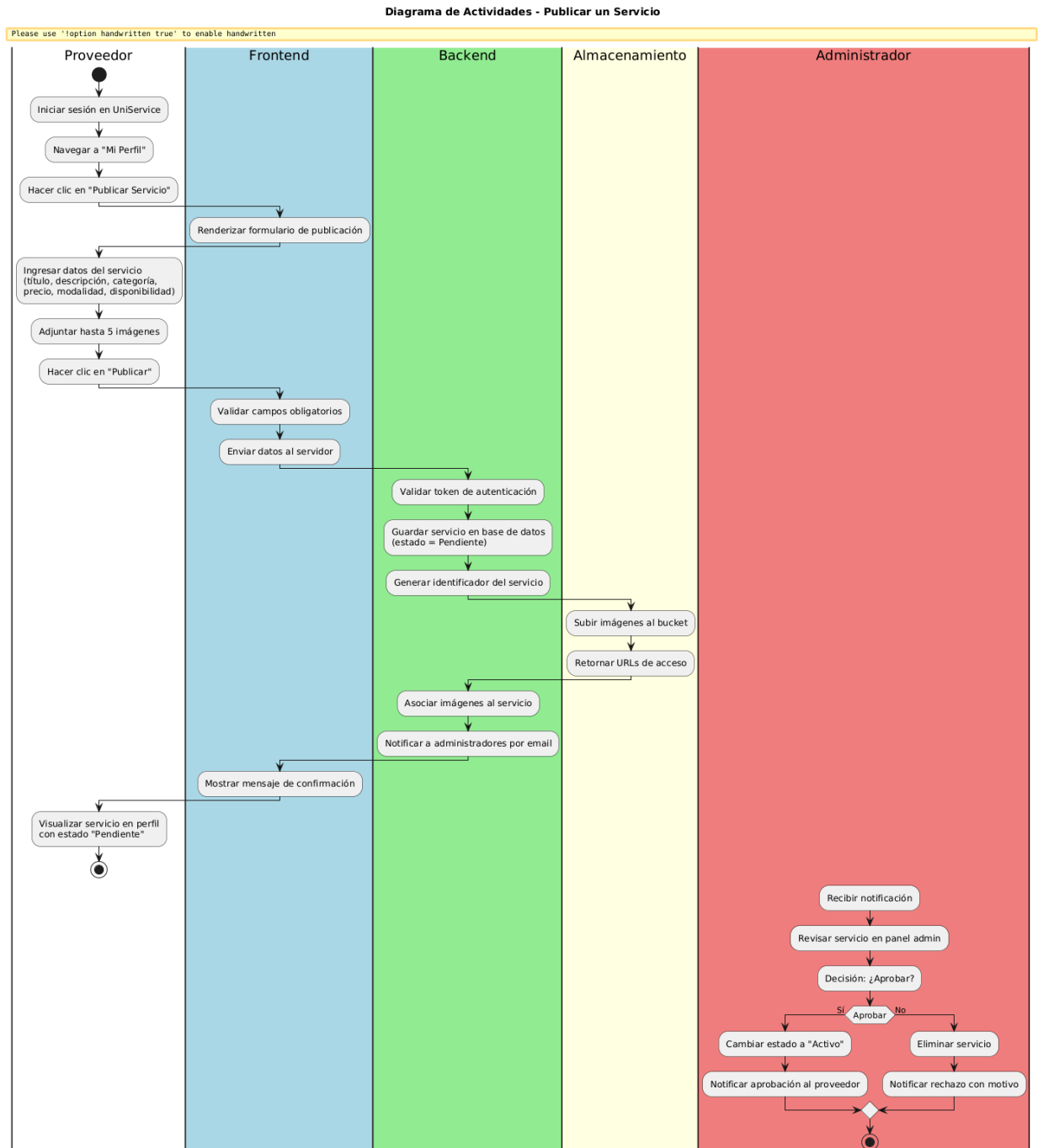


Figura 3. Diagrama de Actividades – Solicitar un Servicio

Diagrama de Actividades - Solicitar un Servicio

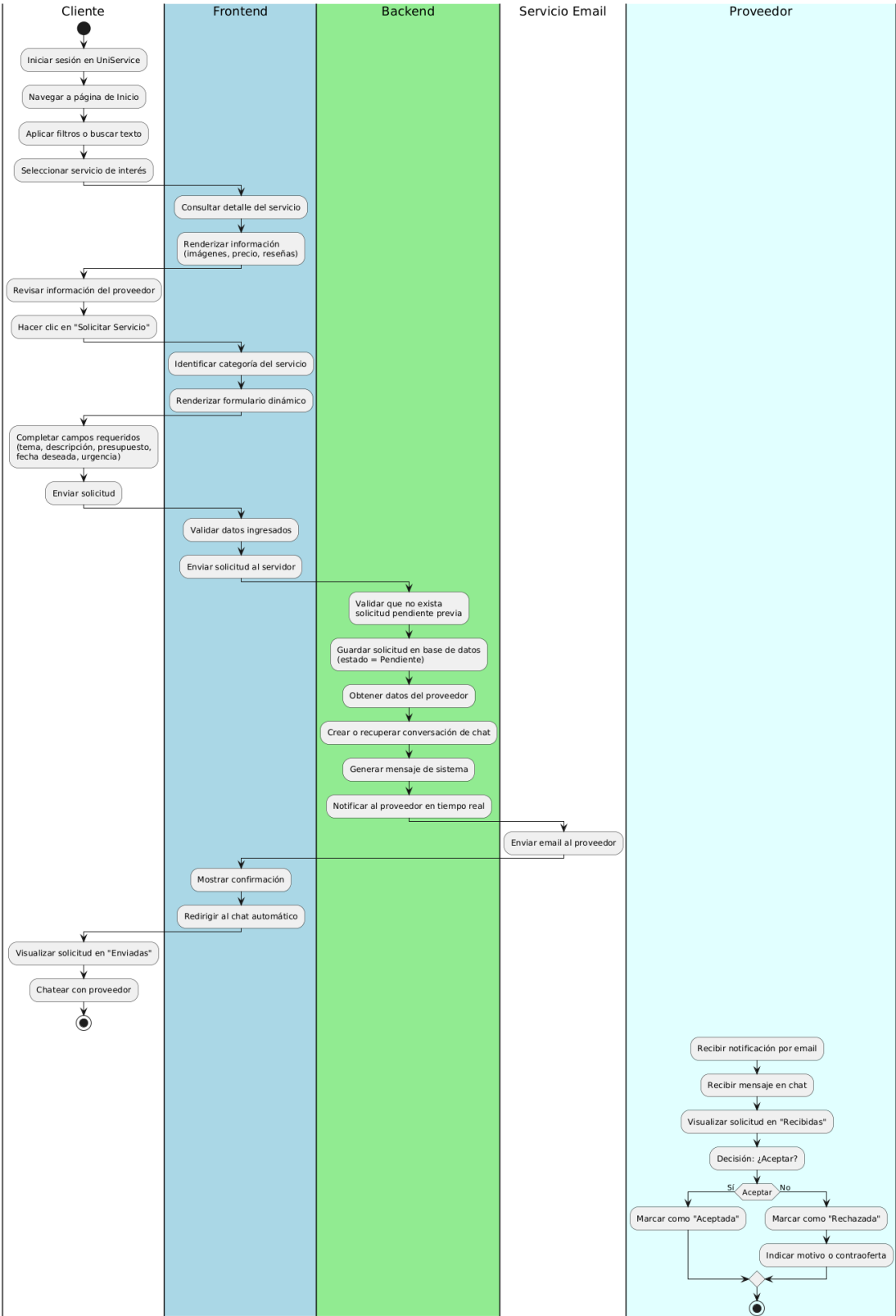


Figura 4. Diagrama de Actividades – Proceso de Reporte

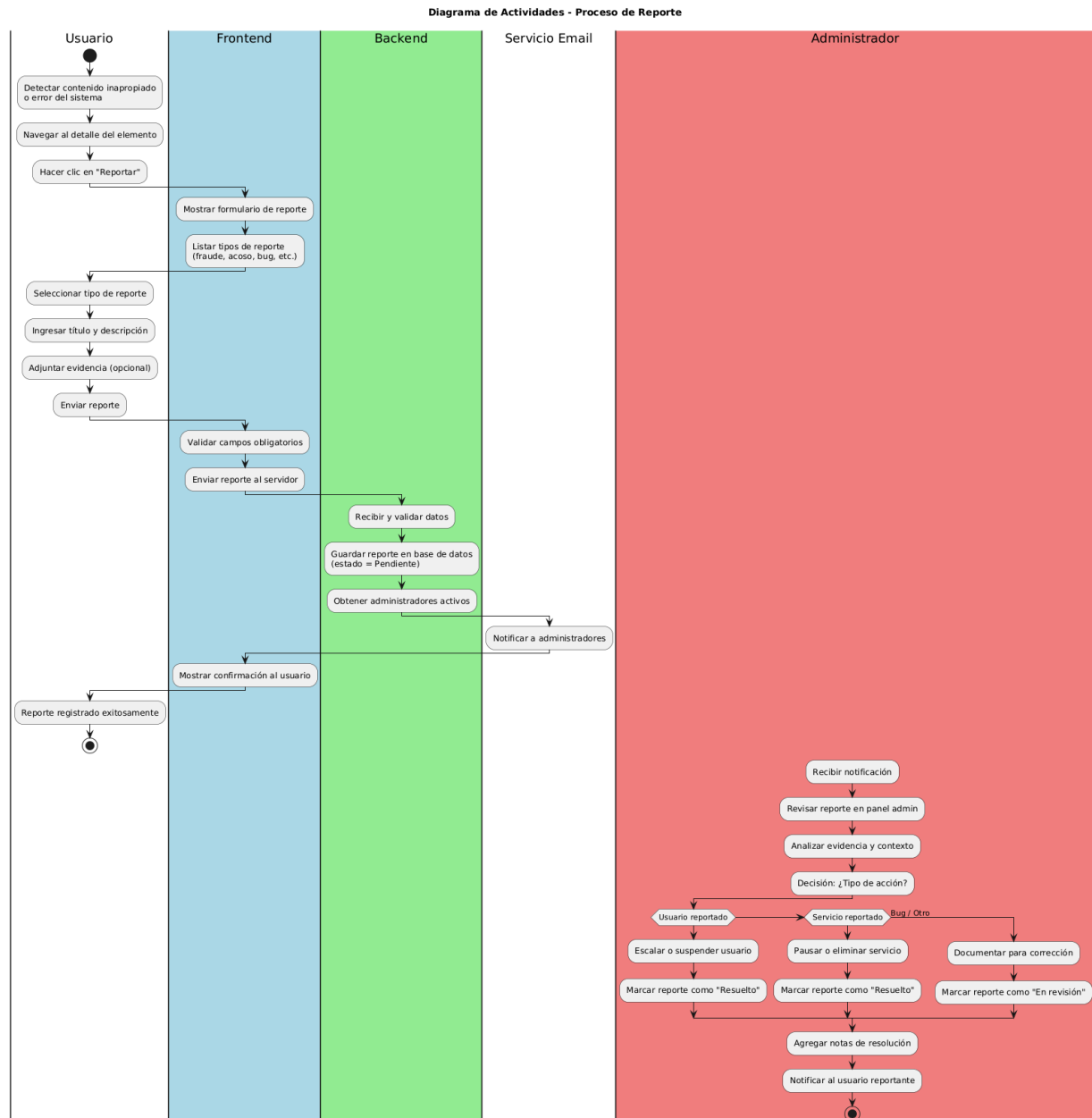


Figura 5. Diagrama de Clases UML – UniService

Diagrama de Clases UML - UniService

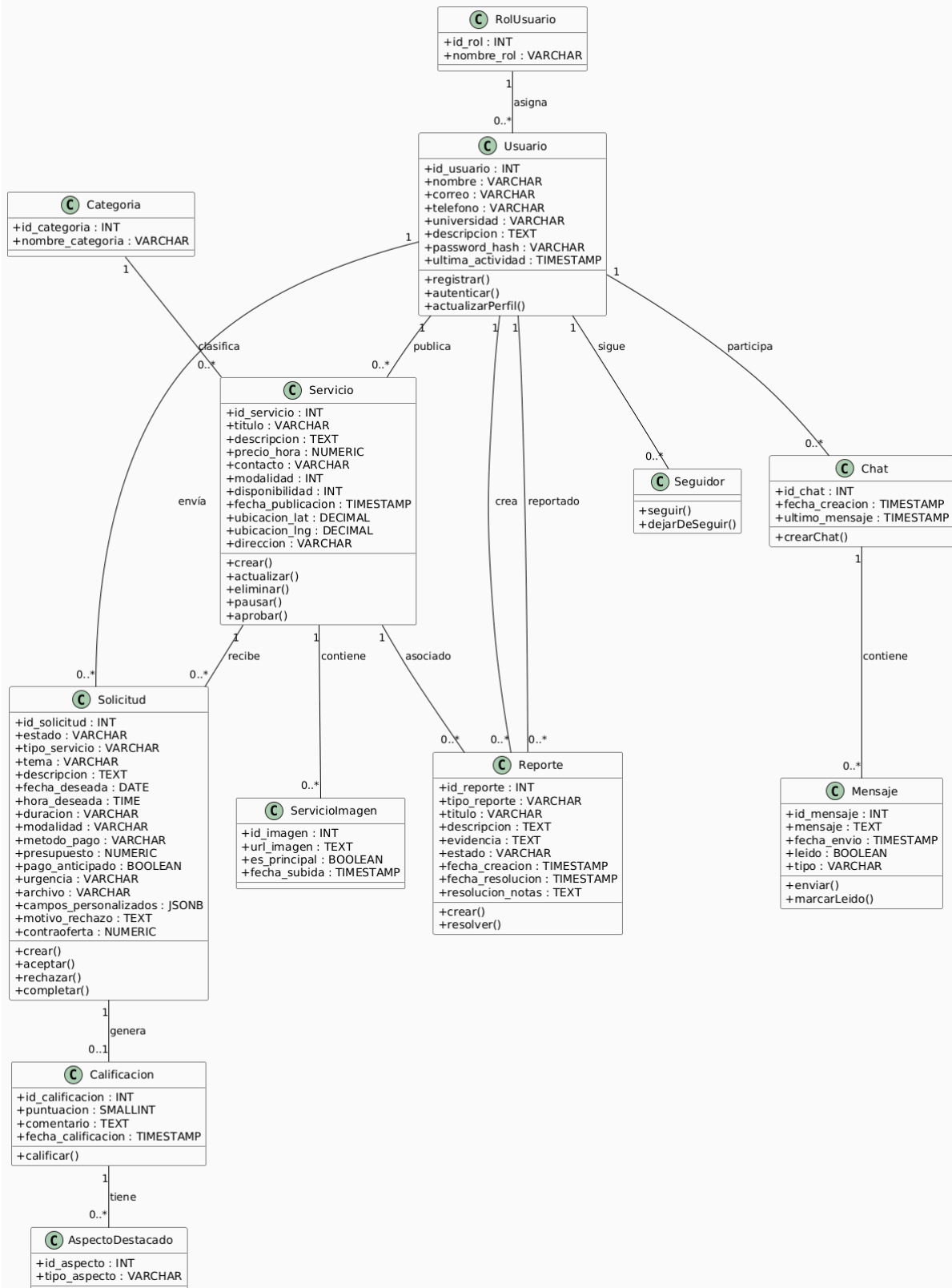


Figura 6. Diagrama de Secuencia – Autenticación de Usuario

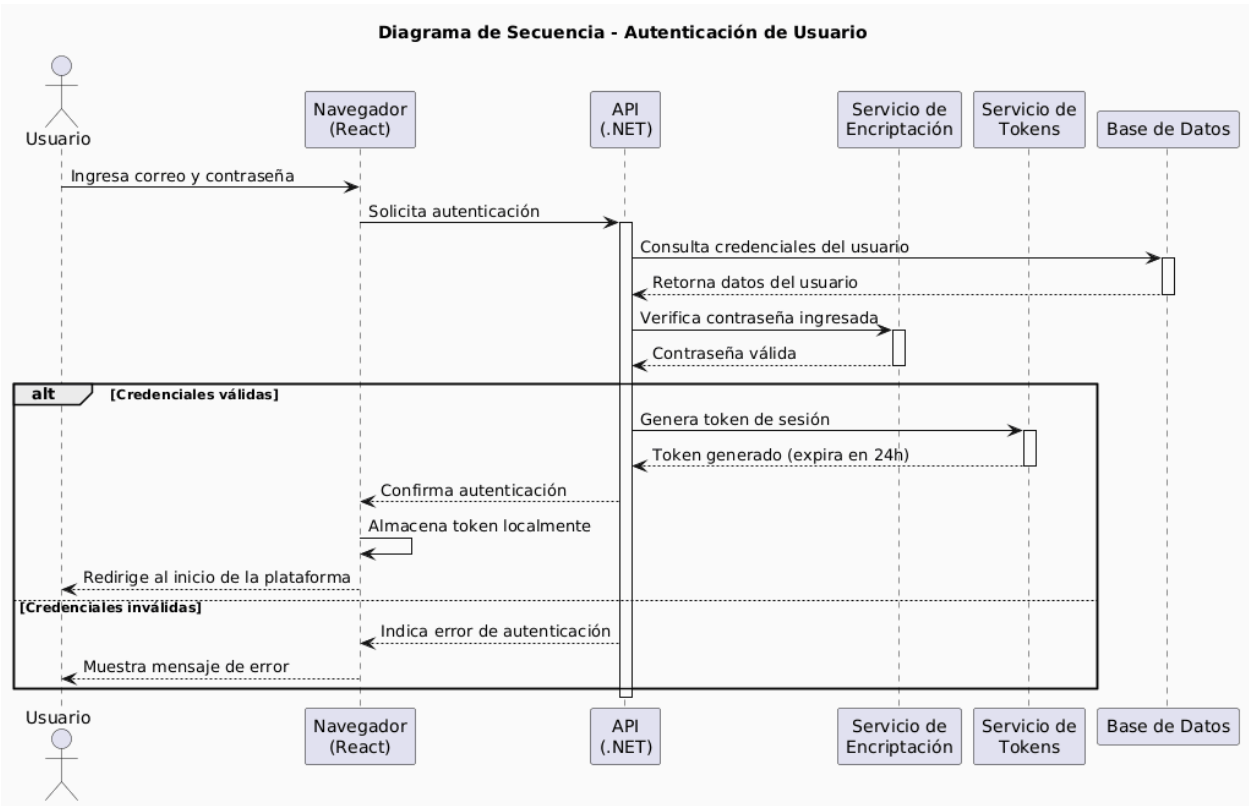


Figura 7. Diagrama de Secuencia – Creación de Solicitud de Servicio

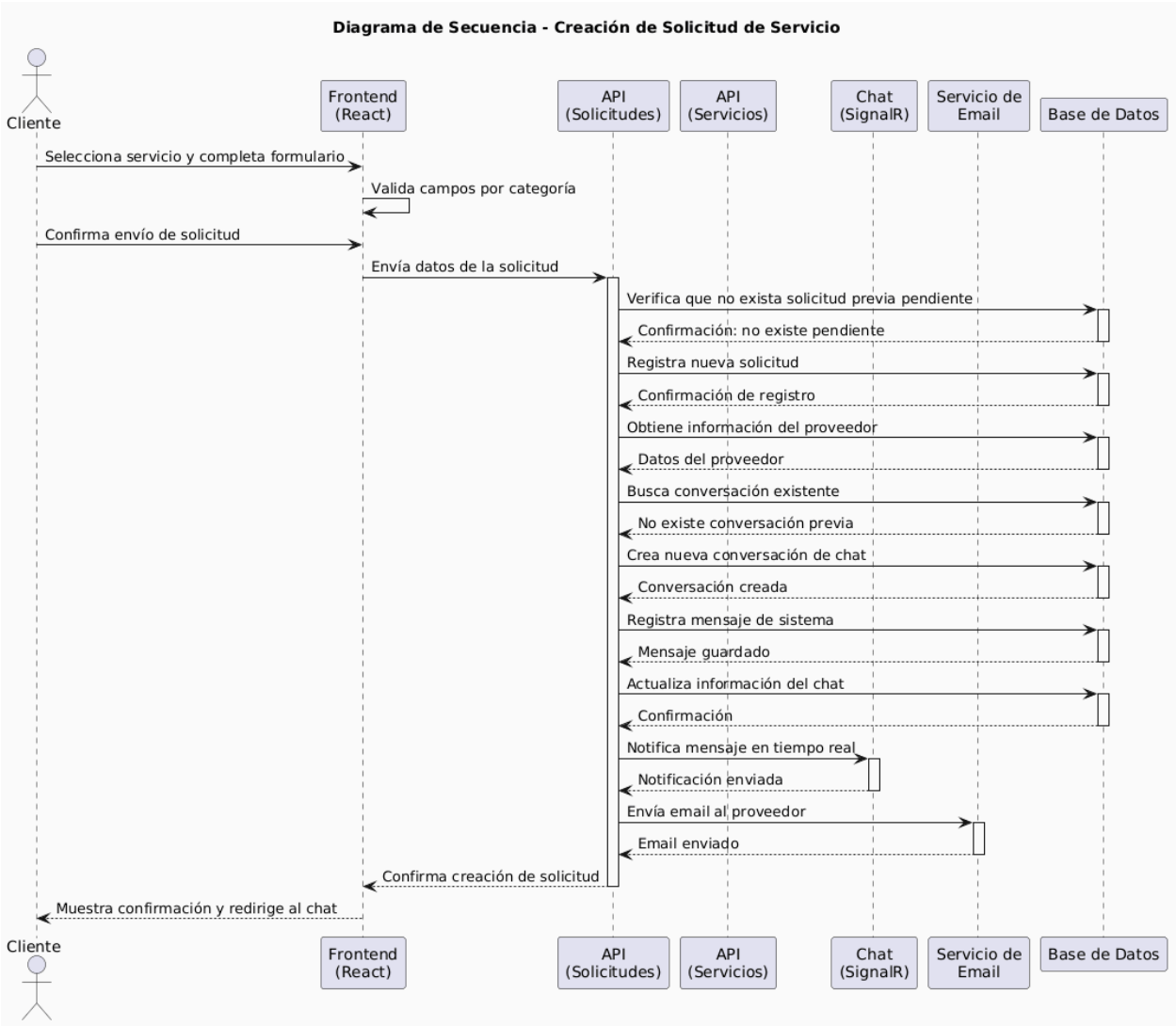


Figura 8. Diagrama de Despliegue UML – UniService

